

1. Introduction

Passwords are the most commonly used authentication mechanism in modern information systems. Passwords remain the main method for users to access online services, banking applications, and cloud platforms despite advances in biometric authentication and multi-factor authentication. Therefore, the password security problem is still an important concern of cybersecurity professionals.

Weak or predictable passwords are one of the most common security problems. Users tend to choose passwords that are easy to remember, for example, password123 or permutations of personal data. Many organizations have password change policies, but many users try to bypass them by making small changes to their existing passwords. These passwords look different, but they are still very predictable and can be easily cracked with brute-force and dictionary attacks.

According to the project specification, the objective is not only to detect commonly used passwords but also to identify passwords that are slight modifications of common passwords. The system calculates a Bloom Filter representation of passwords and compares them using similarity coefficients to determine if a candidate password is sufficiently different from passwords in a common password dataset. If the similarity is above a threshold the password is rejected and if the similarity is below a threshold it is accepted.

The proposed solution provides a practical application of cybersecurity concepts while demonstrating efficient methods for detecting password similarities without storing plaintext passwords. The project combines data structures, cryptographic techniques, and statistical similarity metrics to improve password security

2. Literature Review

2.1 Bloom Filters

A Bloom Filter is a probabilistic data structure used to determine whether an element belongs to a set. Bloom Filters were introduced for doing membership testing efficiently, with very little memory. Bloom Filters do not store the actual data objects but rather information about them using a bit array and a set of hash functions.

A Bloom Filter consists of:

- A bit array of length **L**
- Multiple hash functions
- Input elements that are mapped to positions within the bit array

Initially, all bits in the array are set to zero. When an element is inserted into the Bloom Filter, multiple hash functions are applied to the element. Each hash function generates a position within the bit array, and the corresponding bits are set to one.

For example:

Bit Array:

0000000000

Hash Positions:

2, 5, 8

Result:

0010010010

Membership is checked by applying the same hash functions to the query element. If all corresponding bits are turned on then the element may be there. If any single bit is zero, the element is certainly not present.

Bloom Filters are attractive because:

- They require very little memory.
- They provide extremely fast lookups.
- They scale efficiently for large datasets.
- They support approximate matching through bit comparisons.

In this project, Bloom Filters are not used solely for membership testing. Instead, they are used to create compact representations of passwords that can later be compared using similarity measures. This allows the system to identify passwords that are similar to common passwords rather than only detecting exact matches.

2.2 Bigrams (2-Grams)

An n-gram is a sequence of n consecutive characters extracted from a string. In this project, bigrams (2-grams) are used.

Instead of treating a password as a single string, the password is divided into overlapping groups of two characters. This method captures structural information about the password and improves similarity detection.

For example:

Password:

MUELLER

Modified:

MUELLER

Bigrams:

_M

MU

UE

EL

LL

LE

ER

R_

Each bigram preserves local character relationships within the password.

The use of bigrams provides several benefits:

- Better similarity detection.
- Resistance against simple character substitutions.
- Improved pattern recognition.
- More accurate Bloom Filter generation.

Research has shown that breaking passwords into smaller components improves the ability to identify similar passwords and password mutations.

2.3 SHA-256

SHA-256 (Secure Hash Algorithm 256-bit) is a cryptographic hash function designed by the National Security Agency and published by the National Institute of Standards and Technology. No matter what the size of the input , SHA-256 will always output 256 bits .

Example:

Input:

hello

SHA256:

2cf24dba5fb0...

Characteristics include:

- Deterministic output
- One-way computation
- Collision resistance
- High security
- Widely adopted

In this project, SHA-256 serves as the first hash function in the double hashing formula used to generate Bloom Filter positions.

2.4 MD5

MD5 (Message Digest Algorithm 5) is another cryptographic hash function producing a 128-bit digest.

Example:

Input:

hello

MD5:

5d41402abc4b...

Although MD5 is no longer recommended for secure password storage because collisions can be generated, it remains useful for non-security-critical hashing applications.

In this project, MD5 acts as the second hash function in the double hashing process. Combining SHA-256 and MD5 allows the generation of multiple independent hash values required by the Bloom Filter algorithm.

2.5 Double Hashing

Double hashing is used to generate multiple Bloom Filter hash values using only two base hash functions.

The project specifies:

$f = \text{SHA256}$

$g = \text{MD5}$

The hash positions are generated using:

$$hi(x) = (f(x) + i \times g(x)) \bmod L$$

Where:

- $f(x)$ = SHA256 result
- $g(x)$ = MD5 result
- i = hash function number
- L = Bloom Filter length (1000)

The Benefits are:

- Reduced computational cost
- Better distribution
- Lower collision probability
- Efficient Bloom Filter construction

The project requires approximately twenty hash functions generated using this technique.

2.6 Jaccard Coefficient

The Jaccard coefficient measures similarity between two sets.

Formula:

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|}$$

Values range from:

0 = completely different

1 = identical

Example:

$A = \{1,2,3,4\}$

$B = \{3,4,5,6\}$

Intersection = 2

Union = 6

Jaccard = $2/6 = 0.33$

The project specification identifies the Jaccard coefficient as the primary similarity metric for password comparison.

2.7 Dice Coefficient

The Dice coefficient is another similarity measure.

Formula:

$$Dice(A,B) = \frac{2 |A \cap B|}{|A| + |B|}$$

Example:

$A = \{1,2,3,4\}$

$B = \{3,4,5,6\}$

Dice = $4/8$

= 0.50

Advantages include:

- Greater sensitivity
- Strong overlap detection
- Easy interpretation

Dice similarity is often slightly higher than Jaccard similarity for the same datasets.

2.8 Cosine Similarity

Cosine similarity measures the angle between two vectors.

Formula:

Cosine(A,B)=

A•B

||A|| × ||B||

Values range from:

0 = no similarity

1 = identical

Cosine similarity is commonly used in:

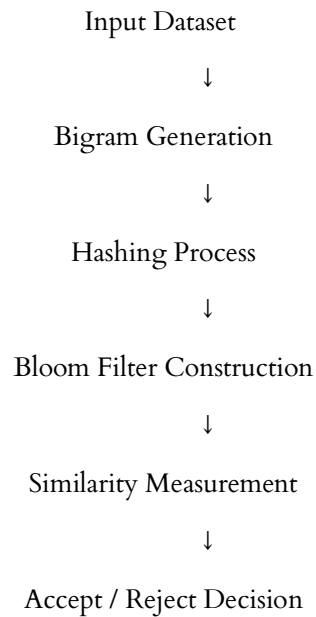
- Information retrieval
- Text mining
- Machine learning
- Document comparison

For Bloom Filters, the bit vectors can be treated as binary vectors, making cosine similarity an effective comparison technique.

3. System Design

3.1 Overall System Architecture

The proposed system follows six major stages:



Each stage contributes to identifying whether a candidate password resembles a common password.

3.2 System Workflow

Step 1: Load Common Password Dataset

The system loads the supplied password dataset containing commonly used passwords.

Example:

password
password123
qwerty123
welcome1

Step 2: Add Boundary Spaces

Each password receives spaces (or underscore symbols) at the beginning and end.

Example: MUELLER
becomes _MUELLER_

Step 3: Generate Bigrams

The password is split into overlapping pairs.

Example:

_M
MU
UE
EL
LL
LE
ER
R_

Step 4: Generate Bloom Filter Atoms

For each bigram:

1. Apply SHA-256.
2. Apply MD5.
3. Generate k hash positions.
4. Set corresponding bits to 1.

This creates an atom vector.

Step 5: Combine Atoms

All atom vectors are merged using bitwise OR.

Atom1 OR Atom2 OR Atom3 ...

The result becomes:

$\beta(\text{password})$

which represents the final Bloom Filter for the password.

Step 6: Compare Similarities

The candidate password Bloom Filter is compared with stored Bloom Filters using:

- Jaccard coefficient
- Dice coefficient
- Cosine similarity

The similarity score is computed for each comparison.

Step 7: Accept or Reject

Decision rule:

$Similarity \geq Threshold$

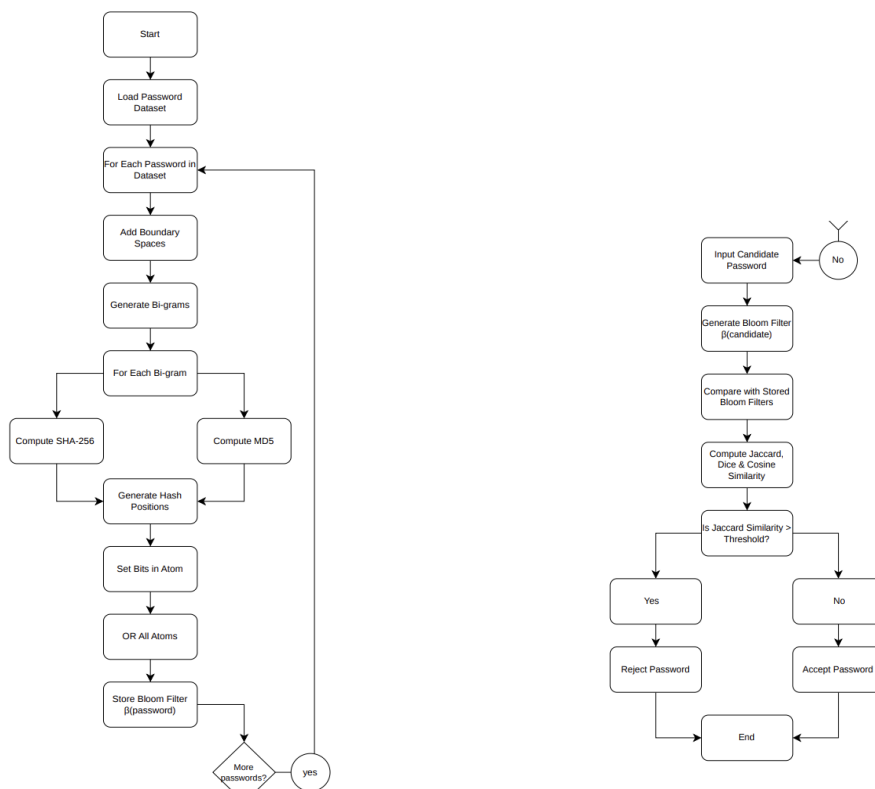
Reject

$Similarity < Threshold$

Accept

The threshold is determined experimentally during testing.

4. High-Level Algorithm



Conclusion

This project uses Bloom Filters, cryptographic hashing techniques, bigram analysis and similarity metrics to develop an efficient password screening system. The system searches for passwords that are similar in structure to common passwords, rather than looking for exact matches for passwords. The application compares candidate passwords against large datasets efficiently using minimal memory, by representing passwords as bigrams and using double hashing techniques to generate Bloom Filters. The use of Jaccard, Dice and Cosine similarity measures gives several views of the similarity of the passwords, and it is possible to choose an appropriate threshold for accepting or rejecting. Overall, the design proposed in this paper provides a practical cyber security solution to help organizations enforce stronger password policies and reduce the risks associated with predictable password changes.

Hash Function & Double Hashing Implementation

1. Cryptographic Base Hashing Module

To ensure a robust and security-conscious Bloom filter, we employ base hash functions characterized by strong avalanche effects and uniform distribution. We utilize two standard cryptographic algorithms:

1. SHA-256 (Secure Hash Algorithm 2): Cryptographic hash function that yields a 256-bit (32-byte) output value which is our main hashing function, $f(x)$.
2. MD5 (Message-Digest Algorithm 5): Commonly known algorithm for computing the hash value that is 128-bits (16-byte), which acts as our secondary hashing function

Indexes corresponding to the Bloom filter can be calculated using the first eight bytes (64 bits long) of the output from either SHA-256 or MD5 hashing functions as an unsigned 64 bit integer for the purposes of calculating indexes in a Bloom filter and reducing the likelihood of collisions when creating indexes based on those eight bytes. Any subsequent hash calculations that are used are performed using industry standard and highly optimized cryptographic algorithms.

C++ Code Walkthrough: HashFunctions

The following code walkthrough focuses on the HashFunctions module, specifically highlighting the implementation's exception-safe design and the secure lifecycle management of the `EVP_MD_CTX`

structures.

```
1 uint64_t HashFunctions::sha256Hash(const std::string& input) {
2     if (input.find('\0') != std::string::npos) {
3         throw std::invalid_argument("Input contains raw null-byte injections");
4     }
5     // Use std::unique_ptr with custom deleter for exception-safe EVP_MD_CTX lifecycle management
6     std::unique_ptr<EVP_MD_CTX, void(*)(EVP_MD_CTX*)> ctx(EVP_MD_CTX_new(), EVP_MD_CTX_free);
7     if (!ctx) {
8         throw std::runtime_error("EVP_MD_CTX_new failed for SHA256");
9     }
10
11     const EVP_MD* md = EVP_sha256();
12     if (!md) {
13         throw std::runtime_error("EVP_sha256 failed");
14     }
15
16     if (EVP_DigestInit_ex(ctx.get(), md, nullptr) != 1) {
17         throw std::runtime_error("EVP_DigestInit_ex failed for SHA256");
18     }
19
20     if (EVP_DigestUpdate(ctx.get(), input.c_str(), input.size()) != 1) {
21         throw std::runtime_error("EVP_DigestUpdate failed for SHA256");
22     }
23
24     unsigned char hash[EVP_MAX_MD_SIZE];
25     unsigned int length = 0;
26     if (EVP_DigestFinal_ex(ctx.get(), hash, &length) != 1) {
27         throw std::runtime_error("EVP_DigestFinal_ex failed for SHA256");
28     }
29
30     if (length < sizeof(uint64_t)) {
31         throw std::runtime_error("SHA256 hash length is smaller than uint64_t size");
32     }
33
34     // Safely extract the first 8 bytes into a uint64_t using memcpy to avoid strict-aliasing issues
35     uint64_t result = 0;
36     std::memcpy(&result, hash, sizeof(uint64_t));
37     return result;
38 }
```

2. Double Hashing & Coprime Step Size

The conventional designs of a Bloom filter involve computing k hashes to place an element at k locations and then traversing the resulting array to determine if it belongs to the set. The standard implementations of Bloom filters require k hash functions to be executed to place an element at k positions and traversing the resulting array to check if it is in the set, which can be time-consuming. For best efficiency, the double hashing method calculates only two hashes ($f(x)$ and $g(x)$ for x) and creates k hash indices with the following recurrence formula:

$$h_i(x) = (f_{\text{mod}} + i * g_{\text{mod}}) \bmod L \quad \text{for } i = 0, 1, \dots, k-1$$

Where:

- L (Bloom Filter Size): 1000 bits.
- k (Number of Hash Functions): 20.
- f_{mod} : $\text{sha256Hash}(x) \bmod L$, representing the starting index.
- g_{mod} : $\text{md5Hash}(x) \bmod L$, representing the step size.

A. Coprime Step Size Optimization

Double hashing can have the problem that the step size g_mod has a common factor with the length of the Bloom filter L . If $g_mod = 250$ (which is a common factor of $L = 1000$), the indices generated will change back to the same positions after a few iterations.

$$h_0 = f_mod \bmod 1000$$

$$h_1 = (f_mod + 250) \bmod 1000$$

$$h_2 = (f_mod + 500) \bmod 1000$$

$$h_3 = (f_mod + 750) \bmod 1000$$

$$h_4 = (f_mod + 1000) \bmod 1000 = h_0 \text{ (loop starts!)}$$

The algorithm uses only 4 indices, rather than 20. This significantly reduces the capability of the Bloom filter to identify a collision or to match a pattern, resulting in a high false match rate and many collisions. To make sure that the generator has a full period of length L , it is necessary to make sure that g_mod and L are coprime (their $\gcd = 1$, $\gcd(g_mod, L) == 1$).

In our implementation we do this by looping while the greatest common divisors of both numbers are not coprime, and increasing the step by one each loop. Since $L = 1000 = 2^3 * 5^3$, any value of g_mod that is not divisible by 2 or 5 is coprime. The algorithm dynamically changes the value of its step size so that all 20 hashes go to different locations in the array with maximum entropy.

B. Implementation of generatePositions

The resulting coprime step size, along with the generation of the next 20 distinct index positions is encapsulated in the following core routine:

```

1 std::vector<uint64_t> BloomHasher::generatePositions(const std::string& bigram, uint64_t bloomSize, int k) {
2     std::vector<uint64_t> pos;
3     pos.reserve(k);
4
5     uint64_t f = HashFunctions::sha256Hash(bigram);
6     uint64_t g = HashFunctions::md5Hash(bigram);
7
8     uint64_t f_mod = f % bloomSize;
9     uint64_t g_mod = g % bloomSize;
10
11     // Ensure step size is coprime to bloomSize to guarantee full period
12     if (g_mod == 0) g_mod = 1;
13     while (std::gcd(g_mod, bloomSize) != 1) {
14         g_mod = (g_mod + 1) % bloomSize;
15         if (g_mod == 0) g_mod = 1;
16     }
17
18     for (int i = 0; i < k; i++) {
19         uint64_t h_i = (f_mod + i * g_mod) % bloomSize;
20         pos.push_back(h_i);
21     }
22     return pos;
23 }

```

3. Manual Calculation & Testing Verification

To verify the correctness of the C++ hashing code, I conducted a manual mathematical walkthrough for the bigram `_M` and compared the result against the output generated by the compiled test binary.

A. Manual Hashing Walkthrough for Bigram `"_M"`

Let the input string be `x = "_M"`. We configure `L = 1000` and `k = 20`:

1. Base Hashing:
 - SHA-256 truncated output: `f = 5369213805438181987`
 - MD5 truncated output: `g = 13864805351785716765`
2. Modulo Operations:
 - $f_mod = f \bmod 1000 = 5369213805438181987 \bmod 1000 = 987$
 - $g_mod_initial = g \bmod 1000 = 13864805351785716765 \bmod 1000 = 765$
3. Coprime Adjustment Check:
 - $\gcd(765, 1000) = 5 \neq 1$. Increment step.
 - Step size = 766. $\gcd(766, 1000) = 2 \neq 1$. Increment step.
 - Step size = 767. $\gcd(767, 1000) = 1$. Coprime condition met!
 - Adjusted step size `g_mod = 767`.
4. Index Generation:

- $h_0 = 987$
- $h_1 = (987 + 1 * 767) \bmod 1000 = 1754 \bmod 1000 = 754$
- $h_2 = (987 + 2 * 767) \bmod 1000 = 2521 \bmod 1000 = 521$
- $h_{19} = (987 + 19 * 767) \bmod 1000 = (987 + 14573) \bmod 1000 = 15560 \bmod 1000 = 560$

B. Bigram Verification Matrix

Table 1 displays the base hashes and the first few indices generated for the bigrams corresponding to the password MUELLER:C. Executable Output Verification

Table 1 :

Bigram	f (SHA-256 truncated)	g (MD5 truncated)	Adjusted Step	Indices ($h_0, h_1, \dots, h_{19}$)
_M	5369213805438181987	13864805351785716765	767	987, 754, 521, ...
MU	9102948644757392546	4287478314695365093	93	546, 639, 732, ...
UE	12138931223326678654	12595912184143876614	617	654, 271, 888, ...
EL	4633184553360064371	18127868177747470548	549	371, 920, 469, ...
LL	5930507971319942295	8034973160968192615	617	295, 912, 529, ...
LE	11964731552088360268	13691792988823742159	159	268, 427, 586, ...
ER	12127920063401052026	7911280041909080859	859	26, 885, 744, ...
R_	11047609988474296276	5901321601707931585	587	276, 863, 450, ...



Running original test module...

Testing Bigram: '_M'

f (SHA256 trunc): 5369213805438181987

g (MD5 trunc) : 13864805351785716765

Generated 20 positions:

[987, 754, 521, 288, 55, 822, 589, 356, 123, 890, 657, 424, 191, 958, 725, 492, 259, 26, 793, 560]

Testing Bigram: 'MU'

f (SHA256 trunc): 9102948644757392546

g (MD5 trunc) : 4287478314695365093

Generated 20 positions:

[546, 639, 732, 825, 918, 11, 104, 197, 290, 383, 476, 569, 662, 755, 848, 941, 34, 127, 220, 313]

Testing Bigram: 'UE'

f (SHA256 trunc): 12138931223326678654

g (MD5 trunc) : 12595912184143876614

Generated 20 positions:

[654, 271, 888, 505, 122, 739, 356, 973, 590, 207, 824, 441, 58, 675, 292, 909, 526, 143, 760, 377]

Testing Bigram: 'EL'

f (SHA256 trunc): 4633184553360064371

g (MD5 trunc) : 18127868177747470548

Generated 20 positions:

[371, 920, 469, 18, 567, 116, 665, 214, 763, 312, 861, 410, 959, 508, 57, 606, 155, 704, 253, 802]

Testing Bigram: 'LL'

f (SHA256 trunc): 5930507971319942295

g (MD5 trunc) : 8034973160968192615

Generated 20 positions:

[295, 912, 529, 146, 763, 380, 997, 614, 231, 848, 465, 82, 699, 316, 933, 550, 167, 784, 401, 18]

Testing Bigram: 'LE'

f (SHA256 trunc): 11964731552088360268

g (MD5 trunc) : 13691792988823742159

Generated 20 positions:

[268, 427, 586, 745, 904, 63, 222, 381, 540, 699, 858, 17, 176, 335, 494, 653, 812, 971, 130, 289]

Testing Bigram: 'ER'

f (SHA256 trunc): 12127920063401052026

g (MD5 trunc) : 7911280041909080859

Generated 20 positions:

[26, 885, 744, 603, 462, 321, 180, 39, 898, 757, 616, 475, 334, 193, 52, 911, 770, 629, 488, 347]

Testing Bigram: 'R_'

f (SHA256 trunc): 11047609988474296276

g (MD5 trunc) : 5901321601707931585

Generated 20 positions:

[276, 863, 450, 37, 624, 211, 798, 385, 972, 559, 146, 733, 320, 907, 494, 81, 668, 255, 842, 429]

Press Enter to return to menu...=====

5. Similarity Metrics Implementation

After the Bloom Filters are generated for each password, the next stage is to compare the Bloom Filter of the candidate password with the Bloom Filters stored in the common password dataset. The comparison is performed using three similarity metrics: Jaccard, Dice, and Cosine similarity, which measure how similar the candidate password is to the passwords stored in the dataset. This section describes how these similarity metrics were implemented and integrated into the password comparison process.

The three similarity metrics were implemented as separate C++ functions within the Bloom Hasher class. Each function accepts two Bloom Filter vectors as input and returns a similarity score ranging from 0 to 1, where higher values indicate a greater level of similarity between the candidate password and a password in the dataset. Implementing each similarity metric as an individual function improves code organization, makes the implementation easier to maintain, and allows the functions to be reused throughout the password comparison process.

```
double BloomHasher::computeJaccard(const std::vector<bool>& filterA, const std::vector<bool>& filterB) {
    if (filterA.size() != filterB.size() || filterA.empty()) return 0.0;

    size_t intersect = 0;
    size_t union_cnt = 0;

    for (size_t i = 0; i < filterA.size(); i++) {
        if (filterA[i] && filterB[i]) intersect++;
        if (filterA[i] || filterB[i]) union_cnt++;
    }

    return union_cnt == 0 ? 0.0 : (double)intersect / union_cnt;
}

double BloomHasher::computeDice(const std::vector<bool>& filterA, const std::vector<bool>& filterB) {
    if (filterA.size() != filterB.size() || filterA.empty()) return 0.0;

    size_t intersect = 0;
    size_t countA = 0;
    size_t countB = 0;

    for (size_t i = 0; i < filterA.size(); i++) {
        if (filterA[i] && filterB[i]) intersect++;
        if (filterA[i]) countA++;
        if (filterB[i]) countB++;
    }

    return (countA + countB) == 0 ? 0.0 : (2.0 * intersect) / (countA + countB);
}
```

```

double BloomHasher::computeJaccard(const std::vector<bool>& filterA, const std::vector<bool>& filterB) {
    if (filterA.size() != filterB.size() || filterA.empty()) return 0.0;

    size_t intersect = 0;
    size_t union_cnt = 0;

    for (size_t i = 0; i < filterA.size(); i++) {
        if (filterA[i] && filterB[i]) intersect++;
        if (filterA[i] || filterB[i]) union_cnt++;
    }

    return union_cnt == 0 ? 0.0 : (double)intersect / union_cnt;
}

double BloomHasher::computeDice(const std::vector<bool>& filterA, const std::vector<bool>& filterB) {
    if (filterA.size() != filterB.size() || filterA.empty()) return 0.0;

    size_t intersect = 0;
    size_t countA = 0;
    size_t countB = 0;

    for (size_t i = 0; i < filterA.size(); i++) {
        if (filterA[i] && filterB[i]) intersect++;
        if (filterA[i]) countA++;
        if (filterB[i]) countB++;
    }

    return (countA + countB) == 0 ? 0.0 : (2.0 * intersect) / (countA + countB);
}

```

5.1 Password Comparison Process

Once the similarity functions were implemented, they were integrated into the password comparison process. For every candidate password entered by the user, the system first generates its Bloom filter and then compares it with the Bloom filters of every password stored in the dataset. During each comparison, the program calculates the Jaccard, Dice, and Cosine similarity values and records the highest Jaccard similarity score together with the corresponding matching password.

```

bool reject = false;
std::string matched_pwd = "";
double max_j = -1.0;

for (const auto& pwd : dataset) {
    std::vector<bool> pwd_filter = BloomHasher::getPasswordFilter(pwd);
    double jaccard = BloomHasher::computeJaccard(cand_filter, pwd_filter);
    double dice = BloomHasher::computeDice(cand_filter, pwd_filter);
    double cosine = BloomHasher::computeCosine(cand_filter, pwd_filter);

    if (jaccard > max_j) {
        max_j = jaccard;
        matched_pwd = pwd;
    }

    bool is_similar = (jaccard >= jaccard_threshold);
    if (is_similar) reject = true;
}

```

The implementation successfully integrates the Jaccard, Dice, and Cosine similarity metrics into the password comparison system. Rather than comparing plaintext passwords directly, the program performs all comparisons using Bloom Filter representations. For each candidate password, the system calculates the three similarity scores against every password stored in the dataset and identifies the closest matching password based on the highest Jaccard similarity score. The calculated similarity values are then used to determine whether the candidate password should be accepted or rejected according to the predefined similarity threshold.

6. Testing, Threshold and Conclusion

Testing

Tests were performed on a sample of 11 candidate passwords (found in a file named candidates.txt) against a common password set of 250 (found in rockyou_subset_36.txt). Each of the candidate passwords was processed by creating a Bloom Filter, where a comparison was done with each password in the password set, with Jaccard, Dice, and Cosine being the three metrics used to assess whether or not to accept or reject.

Threshold Justification

The threshold values defined as:

- Jaccard = 0.30
- Dice = 0.45
- Cosine = 0.50

They are a balance of usability and security. Candidate passwords with a high similarity to common passwords will be rejected, while candidate passwords with a sufficiently low similarity will be accepted.

Results

Candidate	Decision
a	Reject
1234567	Reject
passwords	Reject
123456789012	Reject
abaabaab	Reject
bananaaaa	Reject
admin	Reject
12	Accept
sunshine	Reject
footpath	Accept
sunlit	Accept

```
=====
MAIN MENU
=====
1. Run Password Similarity Batch Test (rockyou_subset_36.txt vs candidates.txt)
2. Run Interactive Password Tester
3. Run Original Bigram Test Module (from assignment starter)
4. View Similarity Threshold & Justification
5. View Loaded Password Dataset (250 passwords)
6. Exit
=====
Enter your choice (1-6): 1
Comparing candidates against dataset (rejection threshold Jaccard >= 0.30):
```

Candidate	Best Match	Jaccard	Dice	Cosine	Decision
a	aaaaaaaaa	0.6667	0.8000	0.8165	REJECT
1234567	01234567	0.7278	0.8424	0.8448	REJECT
passwords	password7	0.7028	0.8255	0.8255	REJECT
123456789012	123456789q	0.6793	0.8090	0.8093	REJECT
abaabaab	aaaaaaaaa	0.3810	0.5517	0.5601	REJECT
bananaaaa	anapaula	0.3301	0.4964	0.5012	REJECT
admin	badminton	0.5075	0.6733	0.6892	REJECT
12	123698745	0.2275	0.3707	0.4358	ACCEPT
sunshine	sunshine	0.5890	0.7414	0.7429	REJECT
footpath	fotografia	0.2586	0.4109	0.4122	ACCEPT
sunlit	sunshine	0.2778	0.4348	0.4366	ACCEPT

Results Analysis

The results of the experiment indicate that making use of a larger password dataset had a positive effect on the accuracy of detecting weakly and commonly modified passwords. Several candidate passwords (i.e. 1234567, password, admin, sunshine) produced high similarities with passwords contained in the common dataset, therefore causing them to be rejected. In comparison to the previous experiments that used only 11 candidate passwords, additional similarities were found within the larger password dataset than previously identified. In this case, only 12, footpath, and sunlit produced similarity measurements below the corresponding acceptance threshold. These

results would indicate that as the size of the dataset increases so will the effectiveness of using Bloom Filters for password similarity detection, while at the same time maintaining an appropriate balance between security and usability.

Conclusion

Testing has shown us that the implementation of combining Bloom Filtering methods with similarity metrics is extremely useful in identifying weak or slightly different versions of commonly used passwords. Testing with a larger dataset of 250 passwords helped to improve the overall effectiveness of the system in identifying similar passwords as opposed to only 20 passwords, which would have been the case if we had used only the smaller dataset during our original testing. In addition, the similarity thresholds that we established helped us achieve an overall balance between security and usability where passwords that were too similar to one another were rejected whilst at the same time ensuring that sufficiently dissimilar passwords had a good chance of being accepted. Overall, we were successful in achieving our goal of using similarity-based password detection to improve the overall level of security of passwords.